



ASSIGNMENT – 05 – CO5

(Data Acquisition, Cleaning, Analysis and AI Applications)

COURSE NAME : Python Programming

COURSE CODE : CSA0825

COURSE FACULTY : Hariramachandran T

SUBMITTED BY : G.anushka(1912260668)

GITHUB : https://github.com/anushkapearl21-jpg/CSA0825co5.git

PART A – DATA AND PROGRAM DESIGN

TASK A1 – DESIGN THE DATA ACQUISITION (7 Marks)

Bloom's Level: K3 – Apply

Aim

To design a data acquisition system for collecting air-quality readings from multiple monitoring stations. To identify possible data-quality problems in CSV files, design a suitable relational database structure using SQLite and compare CSV-based data processing with SQL queries.

Algorithm

1. Identify the CSV files received from the air-quality monitoring stations.
2. Read the CSV file using Python's CSV module.
3. Identify the column names and data types.
4. Check whether all required columns are available.
5. Identify missing, duplicate, invalid and inconsistent values.
6. Design a relational database containing station details and air-quality readings.
7. Create primary keys and foreign keys to maintain data integrity.
8. Insert the collected readings into the database.
9. Compare the advantages of CSV files and SQL databases for data analysis.

Code

```
import csv
import io
import sqlite3

data = """station,date,hour,pm25,pm10,no2,temp,unit
Adyar,2026-10-01,08,48,72,18,29,C
Adyar,2026-10-01,09,NA,70,17,30,C
Guindy,2026-10-01,08,95,140,42,86,F
"""

rows = list(csv.DictReader(io.StringIO(data)))

print("Columns:", list(rows[0]))
print("Rows read:", len(rows))
```

```
con = sqlite3.connect(":memory:")

con.executescript("""
CREATE TABLE stations(
    station_id INTEGER PRIMARY KEY,
    station_name TEXT UNIQUE NOT NULL
);

CREATE TABLE readings(
    reading_id INTEGER PRIMARY KEY,
    station_id INTEGER,
    day TEXT,
    hour INTEGER,
    pm25 REAL,
    pm10 REAL,
    no2 REAL,
    temp_c REAL,
    UNIQUE(station_id, day, hour),
    FOREIGN KEY(station_id)
    REFERENCES stations(station_id)
);
""")

con.execute(
    "INSERT INTO stations(station_name) VALUES (?)",
    ("Adyar",)
)

con.execute(
    "INSERT INTO stations(station_name) VALUES (?)",
    ("Guindy",)
)

tables = con.execute("""
SELECT name FROM sqlite_master
WHERE type='table'
ORDER BY name
""").fetchall()

print("Tables created:", [x[0] for x in tables])

count = con.execute(
    "SELECT COUNT(*) FROM stations"
).fetchone()[0]

print("Station count:", count)

con.close()
```

Output

```
Columns: ['station', 'date', 'hour', 'pm25', 'pm10', 'no2', 'temp', 'unit']
Rows read: 3
Tables created: ['readings', 'stations']
Station count: 2
```

Design, Analysis and Observation

A1.1 – Data Acquisition

Data acquisition is the process of collecting information from different sources and making it available for further processing.

In this case study, air-quality monitoring stations collect readings at regular intervals. Each CSV file contains information such as:

- Station name
- Date
- Hour
- PM2.5 concentration
- PM10 concentration
- NO_2 concentration
- Temperature
- Temperature unit

The Python CSV module is used to read the files. The `csv.DictReader()` function reads each row as a dictionary, allowing the program to access values using column names.

A1.2 – Data Quality Problems

The collected CSV files may contain several types of errors.

1. **Missing values:** Some readings may be blank or marked as NA. For example, PM2.5 may be unavailable for a particular hour.
2. **Invalid values:** Negative pollutant values such as -5 are not valid measurements.
3. **Duplicate records:** A station may accidentally upload the same reading more than once.

4. **Inconsistent formats:** Dates may be written in different formats, such as DD-MM-YYYY and YYYY-MM-DD.
5. **Incorrect units:** Some stations may report temperature in Fahrenheit while others report it in Celsius.
6. **Out-of-range values:** Extremely high readings may indicate sensor faults and require validation.
7. **Incorrect data types:** Numeric measurements may sometimes be stored as text.

These problems must be identified before performing statistical analysis.

A1.3 – Database Design

A relational database is useful for storing air-quality information in a structured manner.

The database contains two main tables.

1. Stations Table

Column	Data Type	Description
station_id	INTEGER	Primary key
station_name	TEXT	Name of the station

2. Readings Table

Column	Data Type	Description
reading_id	INTEGER	Primary key
station_id	INTEGER	Foreign key
day	TEXT	Date of reading
hour	INTEGER	Hour of reading
pm25	REAL	PM2.5 measurement
pm10	REAL	PM10 measurement
no2	REAL	NO_2 measurement
temp_c	REAL	Temperature in Celsius

The station ID connects the two tables. The unique constraint on station ID, date and hour prevents duplicate hourly readings.

A1.4 – CSV and SQL Comparison

CSV	SQL Database
Stores data in a text-based format.	Stores data in structured tables.
Easy to create and share.	Supports structured queries.
Repeated analysis requires scanning the file.	Queries can filter and group records efficiently.
Does not directly enforce relationships.	Supports primary keys and foreign keys.
Suitable for data exchange.	Suitable for repeated analysis and storage.

Three examples of queries required by the pollution control board:

1. Calculate the average PM2.5 reading for Adyar during a particular month.

2. Find the maximum daily PM2.5 reading for every station.

3. Count the number of valid readings received from each station.

SQL makes these operations easier using `AVG()`, `MAX()`, `COUNT()` and `GROUP BY`.

Alternative Concept and Complexity Analysis

Another method is to use pandas to read the CSV files.

```
import pandas as pd

df = pd.read_csv("air_quality.csv")

print(df.head())
print(df.info())

print(df.groupby("station")["pm25"].mean())
```

For a dataset containing n records, reading and scanning the CSV file takes approximately O(n) time.

Repeated CSV analysis may require scanning the complete file again. SQL databases can use indexes to speed up repeated searches.

An indexed lookup is approximately O(log n + k), where k is the number of matching records.

Therefore, CSV is suitable for exchanging data, while SQLite is more suitable for structured storage and repeated queries.

TASK A2 – DESIGN THE CLEANING AND NORMALISATION (5 Marks)

Bloom's Level: K4 – Analyse

Aim

To design a data-cleaning process that identifies missing values, invalid measurements, duplicate records and inconsistent units. To apply normalisation techniques to prepare the air-quality data for accurate analysis.

Algorithm

Read the air-quality data.

1. Remove unnecessary spaces from station names.

2. Identify missing values such as blank fields and NA.

3. Convert numeric values into floating-point numbers.

4. Reject negative and out-of-range pollutant values.

5. Remove duplicate station/date/hour records.

6. Convert Fahrenheit temperature into Celsius.

7. Handle missing values using an appropriate strategy.

8. Apply min–max normalisation.

9. Apply z-score standardisation.

10. Store the cleaned values in a consistent format.

Code

```
import statistics

def clean_rows(rows):

    seen = set()
    cleaned = []
    previous = {}

    for station, day, hour, pm25, pm10, no2, temp, unit in rows:

        station = station.strip().title()
        key = (station, day, int(hour))
```

```
        if key in seen:
            continue

        seen.add(key)

        def number(value):
            try:
                return float(value)
            except (TypeError, ValueError):
                return None

        p25 = number(pm25)
        p10 = number(pm10)
        n2 = number(no2)
        t = number(temp)

        if p25 is not None and not 0 <= p25 <= 999:
            p25 = None

        if p10 is not None and not 0 <= p10 <= 2000:
            p10 = None

        if n2 is not None and not 0 <= n2 <= 2000:
            n2 = None

        if t is not None and unit.upper() == "F":
            t = (t - 32) * 5 / 9

        if p25 is None and station in previous:
            p25 = previous[station]

        if p25 is not None:
            previous[station] = p25

        cleaned.append((
            station, day, int(hour), p25,
            p10, n2, round(t, 2) if t is not None else None
        ))

    return cleaned


sample = [
    ("Adyar", "2026-10-01", 8, "48", "72", "18", "29", "C"),
    ("Adyar", "2026-10-01", 9, "NA", "70", "17", "30", "C"),
    ("Adyar", "2026-10-01", 9, "60", "75", "20", "31", "C"),
    ("Guindy", "2026-10-01", 8, "95", "140", "42", "86", "F"),
    ("Guindy", "2026-10-01", 9, "-5", "150", "45", "88", "F")
]

result = clean_rows(sample)

for row in result:
```

```
    print(row)

values = [48.0, 95.0, 121.0]

low = min(values)
high = max(values)

scaled = [
    round((x - low) / (high - low), 3)
    for x in values
]

mean = statistics.mean(values)
sd = statistics.pstdev(values)

z = [
    round((x - mean) / sd, 3)
    for x in values
]

print("Min-max:", scaled)
print("Z-score:", z)
```

Output

```
('Adyar', '2026-10-01', 8, 48.0, 72.0, 18.0, 29.0)
('Adyar', '2026-10-01', 9, 48.0, 70.0, 17.0, 30.0)
('Guindy', '2026-10-01', 8, 95.0, 140.0, 42.0, 30.0)
('Guindy', '2026-10-01', 9, 95.0, 150.0, 45.0, 31.11)
Min-max: [0.0, 0.644, 1.0]
Z-score: [-1.324, 0.232, 1.092]
```

Design, Analysis and Observation

A2.1 – Handling Missing Values

Missing values occur when a sensor fails to record a measurement or when data is lost during transmission.

Common methods for handling missing values include:

- Removing rows with missing values.
- Replacing missing values with the mean.
- Replacing missing values with the median.
- Using forward filling for short gaps in time-series data.

In this program, a missing PM2.5 value is replaced by the previous valid reading from the same station. This method is called forward filling.

Forward filling is suitable for short gaps, but long gaps should be marked as missing because pollution levels may change significantly over time.

A2.2 – Removing Invalid Values

Negative pollutant values are rejected because they are not valid measurements.

The program also applies an upper limit to identify extremely large values that may indicate sensor errors.

Invalid values are stored as `None` so that they can later be represented as SQL NULL.

A2.3 – Removing Duplicate Records

A duplicate occurs when the same station reports multiple readings for the same date and hour.

The program uses the following key:

```
(station, date, hour)
```

A set is used to identify duplicate records. The first record is retained and later duplicate records are skipped.

A2.4 – Unit Conversion

Temperature measurements may be received in Celsius or Fahrenheit.

The conversion formula is:

$C = (F - 32) \times 5/9$

For example:

86°F = 30°C

The program converts Fahrenheit values into Celsius before storing them.

A2.5 – Normalisation

Normalisation is used to bring numerical values into a common scale.

Min–Max Normalisation

Formula:

$x' = (x - xmin) / (xmax - xmin)$

For the values 48, 95 and 121, the normalised values are:

- 48 → 0.000

- 95 → 0.644
- 121 → 1.000

The resulting values fall between 0 and 1.

Z-Score Standardisation

Formula:

$z = (x - \mu) / \sigma$

Where:

- x is the original value.
- μ is the mean.
- σ is the standard deviation.

Z-score standardisation shows how far a value is from the mean in terms of standard deviations.

Min–max normalisation is useful when a fixed range is required. Z-score standardisation is useful when comparing measurements with different scales.

Alternative Concept and Complexity Analysis

Another method for handling missing values is median replacement.

The median is less affected by extreme values than the mean.

A single pass through n records takes O(n) time. Forward filling requires only the previous valid value for each station, so its working memory is small.

Sorting n values to calculate a median takes O(n log n) time.

The choice of cleaning method depends on the number of missing readings, the duration of the gap and the quality of the sensor data.

TASK A3 – DESIGN THE ANALYSIS AND VISUALISATION (5 Marks)

Bloom's Level: K4 – Analyse

Aim

To analyse the cleaned air-quality data and select suitable visualisation techniques for identifying pollution trends, comparing stations, studying relationships and identifying the most polluted hours.

Algorithm

1. Load the cleaned air-quality dataset.
2. Group readings by station and date.
3. Calculate minimum, maximum and average PM2.5 values.
4. Create a line chart to display monthly pollution trends.
5. Create a bar chart to compare stations.
6. Create a scatter plot to study the relationship between temperature and PM2.5.
7. Create a histogram or hourly bar chart to identify pollution patterns by time.
8. Add suitable titles, axis labels and measurement units.
9. Interpret the results and identify important observations.

Code

```
import sqlite3

con = sqlite3.connect(":memory:")

con.execute("""
CREATE TABLE readings(
    station TEXT,
    day TEXT,
    hour INTEGER,
    pm25 REAL,
    temp_c REAL
)
""")

data = [
    ("Adyar", "2026-10-01", 8, 48, 29),
    ("Adyar", "2026-10-01", 9, 52, 30),
    ("Guindy", "2026-10-01", 8, 95, 30),
```

```
    ("Guindy", "2026-10-01", 9, 121, 31),
    ("Adyar", "2026-10-02", 8, 42, 28),
    ("Guindy", "2026-10-02", 8, 110, 32)
]

con.executemany(
    "INSERT INTO readings VALUES (?,?,?,?,?)",
    data
)

daily = con.execute("""
SELECT station, day,
MIN(pm25), MAX(pm25),
ROUND(AVG(pm25), 2), COUNT(pm25)
FROM readings
GROUP BY station, day
ORDER BY station, day
""").fetchall()

for row in daily:
    print(row)

hourly = con.execute("""
SELECT hour, ROUND(AVG(pm25), 2)
FROM readings
GROUP BY hour
ORDER BY AVG(pm25) DESC
""").fetchall()

print("Worst hours first:", hourly)

con.close()
```

Output

```
('Adyar', '2026-10-01', 48.0, 52.0, 50.0, 2)
('Adyar', '2026-10-02', 42.0, 42.0, 42.0, 1)
('Guindy', '2026-10-01', 95.0, 121.0, 108.0, 2)
('Guindy', '2026-10-02', 110.0, 110.0, 110.0, 1)
Worst hours first: [(9, 86.5), (8, 73.75)]
```

Design, Analysis and Observation

A3.1 – Line Chart

A line chart is suitable for displaying changes in pollution levels over time.

- X-axis: Date
- Y-axis: Average PM2.5 concentration

- Purpose: Identify increasing or decreasing pollution trends.

A line chart makes it easy to identify sudden increases in pollution and compare readings across different dates.

A3.2 – Bar Chart

A bar chart is useful for comparing the average pollution levels of different monitoring stations.

- X-axis: Station name
- Y-axis: Average PM2.5 concentration

The sample data shows that Guindy has a higher average PM2.5 reading than Adyar.

A3.3 – Scatter Plot

A scatter plot is used to analyse the relationship between two numerical variables.

For this case study:

- X-axis: Temperature in Celsius
- Y-axis: PM2.5 concentration

The chart can help identify whether higher temperatures are associated with higher or lower PM2.5 readings. However, correlation does not prove causation.

A3.4 – Histogram and Hourly Analysis

A histogram displays the distribution of pollution readings.

An hourly bar chart can show which hours have higher average PM2.5 concentrations.

The sample SQL output indicates that the average reading at hour 9 is higher than at hour 8.

A3.5 – Daily Summary Table

The daily summary contains:

- Station name
- Date
- Minimum PM2.5
- Maximum PM2.5

- Average PM2.5

- Number of valid readings

The SQL query uses `MIN()`, `MAX()`, `AVG()` and `COUNT()` to calculate these values.

Additional Code – Labelled Visualisations

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(
    "cleaned_readings.csv",
    parse_dates=["date"]
)

daily = df.groupby(
    ["station", "date"]
)["pm25"].mean().reset_index()

daily.pivot(
    index="date",
    columns="station",
    values="pm25"
).plot()

plt.title("Daily PM2.5 Trend by Station")
plt.xlabel("Date")
plt.ylabel("Mean PM2.5 (µg/m³)")
plt.tight_layout()
plt.show()

df.groupby("station")["pm25"].mean().plot(kind="bar")

plt.title("Mean PM2.5 by Station")
plt.xlabel("Station")
plt.ylabel("Mean PM2.5 (µg/m³)")
plt.tight_layout()
plt.show()

plt.scatter(df["temp_c"], df["pm25"])

plt.title("Temperature vs PM2.5")
plt.xlabel("Temperature (° C)")
plt.ylabel("PM2.5 (µg/m³)")
plt.tight_layout()
plt.show()

df.groupby("hour")["pm25"].mean().plot(kind="bar")

plt.title("Mean PM2.5 by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Mean PM2.5 (µg/m³)")
plt.tight_layout()
```

```
plt.show()
```

Output

The program generates four figures:

1. Daily PM2.5 Trend by Station.
2. Mean PM2.5 by Station.
3. Temperature vs PM2.5.
4. Mean PM2.5 by Hour.

Each chart includes a title and labelled axes. The figures should be generated using the complete cleaned dataset for meaningful interpretation.

Alternative Concept and Complexity Analysis

The same calculations can be performed using Python dictionaries and loops.

A single pass through n records takes O(n) expected time.

The dictionary stores the aggregate values for each station and date. If there are g station-date groups, the additional memory required is O(g).

SQL `GROUP BY` performs similar aggregation inside the database.

For large datasets, database aggregation is useful because it avoids transferring all raw records into Python.

TASK A4 – PLAN DEBUGGING AND PROFILING (4 Marks)

Bloom's Level: K4 – Analyse

Aim

To understand how debugging and profiling techniques can be used to identify errors and improve the performance of the air-quality data-processing program.

Algorithm

1. Display the first few rows of the raw dataset.
2. Check the input values and data types.
3. Use assertions to verify important conditions.
4. Use logging to record invalid and rejected records.
5. Use a debugger to inspect variables and execution flow.
6. Measure execution time using `time.perf_counter()`.
7. Use `timeit` to compare small operations.
8. Use `cProfile` to identify functions that consume more execution time.

Code

```
import logging
import time
import cProfile
import pstats
import io

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s:%(message)s"
)

def clean(values):

    valid = []

    for row_no, value in enumerate(values, start=1):

        try:
```

```
            number = float(value)

            assert number >= 0, "Negative PM2.5"

            valid.append(number)

        except (ValueError, TypeError) as exc:
            logging.warning(
                "row %d rejected: %s",
                row_no, exc
            )

        except AssertionError as exc:
            logging.warning(
                "row %d rejected: %s",
                row_no, exc
            )

    return valid


raw = ["48", "NA", "-5", "95", "121"]

start = time.perf_counter()

cleaned = clean(raw)

elapsed = time.perf_counter() - start

print("Cleaned:", cleaned)

print(
    "Elapsed seconds:",
    round(elapsed, 6)
)


def workload():
    return clean(raw * 1000)


profile = cProfile.Profile()

profile.enable()

workload()

profile.disable()

stream = io.StringIO()

pstats.Stats(
    profile,
    stream=stream
```

```
).sort_stats("cumulative").print_stats(5)

print("PROFILE")
print(stream.getvalue())
```

Output

```
WARNING:row 2 rejected: could not convert string to float: 'NA'
WARNING:row 3 rejected: Negative PM2.5
Cleaned: [48.0, 95.0, 121.0]
Elapsed seconds: 0.002249
WARNING:row 2 rejected: could not convert string to float: 'NA'
WARNING:row 3 rejected: Negative PM2.5
WARNING:row 7 rejected: could not convert string to float: 'NA'
WARNING:row 8 rejected: Negative PM2.5
WARNING:row 12 rejected: could not convert string to float: 'NA'
WARNING:row 13 rejected: Negative PM2.5
WARNING:row 17 rejected: could not convert string to float: 'NA'
WARNING:row 18 rejected: Negative PM2.5
WARNING:row 22 rejected: could not convert string to float: 'NA'
WARNING:row 23 rejected: Negative PM2.5
WARNING:row 27 rejected: could not convert string to float: 'NA'
WARNING:row 28 rejected: Negative PM2.5
```

The log records rejected values such as:

```
WARNING:row 2 rejected: could not convert string to float: 'NA'

WARNING:row 3 rejected: Negative PM2.5
```

The program also prints:

```
PROFILE
         325001 function calls (323001 primitive calls) in 1.839 seconds

   Ordered by: cumulative time
   List reduced from 101 to 5 due to restriction <5>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      2/1    0.000    0.000    1.839    1.839 /Users/janyaa/Desktop/1St YEAR/Python/ass5.py:267(workload)
        1    0.003    0.003    1.839    1.839 /Users/janyaa/Desktop/1St YEAR/Python/ass5.py:223(clean)
     2000    0.002    0.000    1.836    0.001 /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/logg
ing/__init__.py:2193(warning)
     2000    0.002    0.000    1.834    0.001 /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/logg
ing/__init__.py:1541(warning)
     2000    0.002    0.000    1.832    0.001 /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/logg
ing/__init__.py:1660(_log)
```

Design, Analysis and Observation

A4.1 – Print Statements

Print statements help inspect the program during development.

For example:

```
print(df.head())
print(df.info())
```

They can be used to verify whether the CSV file was loaded correctly.

A4.2 – Assertions

Assertions check whether important conditions are satisfied.

Example:

```
assert pm25 >= 0
```

This checks that a PM2.5 reading is not negative.

Assertions are useful during debugging, but important production data validation should use explicit conditions and error handling.

A4.3 – Logging

Logging records information about program execution.

Common logging levels include:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

Logging is useful for identifying invalid records and tracking failures without depending on print statements.

A4.4 – Debugger

A debugger allows the programmer to:

- Set breakpoints.
- Execute the program step by step.
- Inspect variable values.
- Identify the location of an error.
- Understand the execution flow.

Python provides the `pdb` module for debugging.

A4.5 – Profiling

Profiling measures program performance.

`time.perf_counter()` measures elapsed time.

`timeit` is suitable for comparing small pieces of code.

`cProfile` identifies functions that consume more execution time.

Profiling is important because it helps optimise the actual bottleneck instead of changing code unnecessarily.

Alternative Concept and Complexity Analysis

Print statements are simple but can produce excessive output when processing large files.

Logging provides configurable levels and can store messages in a file.

A debugger is useful during development but is not normally used during regular production execution.

A single pass through n records takes O(n) time. The main objective of profiling is to determine which part of that process consumes the most time.

TASK A5 – DESIGN AN AI APPLICATION FOR CITIZEN COMPLAINTS (4 Marks)

Bloom's Level: K3 – Apply

Aim

To design a simple AI-based system that analyses citizen complaints using sentiment analysis, text summarisation and a rule-based chatbot.

Algorithm

1. Receive complaint text from the citizen.
2. Convert the text into lowercase.
3. Tokenise the complaint into words.
4. Identify positive and negative sentiment words.
5. Calculate the total sentiment score.
6. Apply a simple negation rule.
7. Split the complaint into sentences.
8. Calculate sentence importance using word frequency.
9. Select the most important sentences for summarisation.

10. Extract station names, dates and pollutant names from chatbot questions.

11. Query the database using parameterised SQL statements.

12. Display the result to the citizen.

Code

```
import re
from collections import Counter

POS = {
    "good", "clean", "helpful",
    "improved", "fast"
}

NEG = {
    "bad", "dirty", "rude",
    "slow", "smoke", "broken"
}

NEGATORS = {"not", "never", "no"}


def tokens(text):
    return re.findall(
        r"[a-z]+",
        text.lower()
    )


def sentiment(text):

    words = tokens(text)
    total = 0

    for i, word in enumerate(words):

        value = (
            1 if word in POS
            else -1 if word in NEG
            else 0
        )

        if i and words[i-1] in NEGATORS:
            value *= -1

        total += value

    return total


def summarise(text, limit=2):
```

```
    sentences = re.split(
        r"(?<=[.!?])\s+",
        text.strip()
    )

    words = [
        w for w in tokens(text)
        if len(w) > 3
    ]

    freq = Counter(words)

    scored = []

    for index, sentence in enumerate(sentences):

        score = sum(
            freq[w]
            for w in tokens(sentence)
            if len(w) > 3
        )

        scored.append(
            (score, -index, sentence)
        )

    return [
        s for _, _, s in
        sorted(scored, reverse=True)[:limit]
    ]


complaint = (
    "Roads are clean. "
    "Smoke is bad near school. "
    "Buses are slow."
)

print("Sentiment:", sentiment(complaint))

print("Summary:", summarise(complaint))
```

Output

```
Sentiment: -2
Summary: ['Smoke is bad near school.', 'Roads are clean.']
```

Design, Analysis and Observation

A5.1 – Sentiment Analysis

Sentiment analysis identifies whether a complaint expresses a positive, negative or neutral opinion.

The program uses a small predefined dictionary.

Positive words:

- Good
- Clean
- Helpful
- Improved
- Fast

Negative words:

- Bad
- Dirty
- Rude
- Slow
- Smoke
- Broken

Each positive word contributes +1 and each negative word contributes -1.

A simple negation rule is also applied. For example, the word combination "not helpful" reverses the positive or negative meaning of the following sentiment word.

The final score is used to classify the complaint:

- Score greater than zero: Positive.
- Score less than zero: Negative.
- Score equal to zero: Neutral.

A5.2 – Text Summarisation

Text summarisation reduces a long complaint into a shorter version containing the most important information.

The program uses an extractive summarisation method.

It splits the complaint into sentences, calculates word frequencies and selects the sentences with the highest scores.

Extractive summarisation is useful because the selected sentences come directly from the original complaint.

A5.3 – Chatbot Design

A chatbot can answer questions about air-quality readings.

Example questions:

- What is the PM2.5 level in Adyar?
- What was the average pollution level yesterday?
- Which station recorded the highest reading?
- What is the current air-quality status?

The chatbot identifies the station, date and pollutant from the question and retrieves the corresponding information from SQLite.

The query should use parameter placeholders instead of directly joining user input into SQL.

Limitations

The lexicon method may not correctly understand sarcasm, long sentences, spelling mistakes or multiple languages.

A simple extractive summariser may also miss the main point of a complaint.

A more advanced model can improve language understanding but requires more computing resources and testing.

Alternative Concept and Complexity Analysis

A Naive Bayes classifier can be trained using previously labelled complaints.

A trained model may understand word patterns better than a small fixed dictionary.

The lexicon-based approach takes approximately O(w) time for a complaint containing w words, assuming efficient dictionary lookups.

It requires only a small amount of memory for the predefined word lists.

A trained language model generally requires more memory and processing power but can handle more complex language patterns.

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND VALIDATE THE CLEANING PIPELINE (7 Marks)

Bloom's Level: K3 – Apply

Aim

To implement a complete data-cleaning pipeline that reads CSV data, identifies invalid records, performs normalisation, stores the cleaned data in SQLite and executes SQL queries.

Algorithm

- Read the CSV file using Python's CSV module.
- Check whether the PM2.5 value is missing.
- Convert valid PM2.5 values into floating-point numbers.
- Reject negative values.
- Reject values greater than the selected upper limit of 999.
- Store the valid records in a list.
- Apply min–max normalisation.
- Create a SQLite table.
- Insert the cleaned data into the table.
- Execute SQL queries to calculate station averages and record counts.
- Prepare visualisations using matplotlib.

Code

```
import csv
import io
import sqlite3

RAW = """station,date,pm25
Adyar,2026-10-01,48
```

```
Adyar,2026-10-02,NA
Adyar,2026-10-03,
Guindy,2026-10-01,95
Guindy,2026-10-02,-5
Guindy,2026-10-03,121
Adyar,2026-10-04,1200
"""


def load_clean(text):

    rows = []

    for r in csv.DictReader(io.StringIO(text)):

        raw = r["pm25"].strip()

        if raw == "" or raw.upper() == "NA":
            continue

        try:
            value = float(raw)
        except ValueError:
            continue

        if not 0 <= value <= 999:
            continue

        rows.append((
            r["station"].strip(),
            r["date"],
            value
        ))

    return rows


def min_max(values):

    if not values:
        return []

    low = min(values)
    high = max(values)

    if low == high:
        return [0.0 for _ in values]

    return [
        round((v-low)/(high-low), 3)
        for v in values
    ]
```

```
clean = load_clean(RAW)

print("Cleaned:", clean)

print(
    "Normalised:",
    min_max([pm for _, _, pm in clean])
)

con = sqlite3.connect(":memory:")

con.execute("""
CREATE TABLE aq(
    station TEXT,
    day TEXT,
    pm25 REAL,
    PRIMARY KEY(station, day)
)
""")

con.executemany(
    "INSERT INTO aq VALUES (?,?,?)",
    clean
)

print(
    "SQL rows:",
    con.execute(
        "SELECT COUNT(*) FROM aq"
    ).fetchone()[0]
)

print(
    "Adyar mean:",
    con.execute(
        "SELECT ROUND(AVG(pm25),2) "
        "FROM aq WHERE station=?",
        ("Adyar",)
    ).fetchone()[0]
)

print(
    "Station means:",
    con.execute("""
    SELECT station, ROUND(AVG(pm25),2)
    FROM aq
    GROUP BY station
    """).fetchall()
)

con.close()
```

Output

```
========== RESTART: /Users/janyaa/Desktop/1st YEAR/Python/ass5.py ==========
Cleaned: [('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy', '2026-10-03', 121.0)]
Normalised: [0.0, 0.644, 1.0]
SQL rows: 3
Adyar mean: 48.0
Station means: [('Adyar', 48.0), ('Guindy', 108.0)]
```

Design, Analysis and Observation

B1.1 – Data Cleaning

The raw CSV contains seven records.

The records include:

- Valid readings: 48, 95 and 121.
- Missing value: NA.
- Blank value.
- Negative value: -5.
- Out-of-range value: 1200.

The cleaning function removes missing, negative and out-of-range values.

After cleaning, three valid records remain.

B1.2 – Normalisation

The program applies min–max normalisation.

Formula:

$x' = (x - xmin) / (xmax - xmin)$

For the values 48, 95 and 121, the normalised values are 0.000, 0.644 and 1.000.

This makes it easier to compare measurements using a common scale.

B1.3 – SQLite Storage

The cleaned records are inserted into an SQLite table.

The `executemany()` function inserts multiple records efficiently.

The SQL query calculates the number of stored records and the average PM2.5 reading for each station.

The results show that Adyar has an average reading of 48.0 and Guindy has an average reading of 108.0.

B1.4 – Visualisation

The cleaned dataset can be visualised using:

- Line chart for pollution trends.
- Bar chart for station comparison.
- Scatter plot for temperature and PM2.5 relationship.
- Hourly bar chart for pollution patterns.

Additional Code – Data Visualisation

```
import pandas as pd
import matplotlib.pyplot as plt


df = pd.read_csv("cleaned_readings.csv")


daily = df.groupby(
    ["station", "date"]
)["pm25"].mean().reset_index()


daily.pivot(
    index="date",
    columns="station",
    values="pm25"
).plot()


plt.title("Daily PM2.5 Trend")
plt.xlabel("Date")
plt.ylabel("PM2.5")
plt.show()


df.groupby("station")["pm25"].mean().plot(
```

```
    kind="bar"
)


plt.title("Station Comparison")
plt.xlabel("Station")
plt.ylabel("Average PM2.5")
plt.show()


plt.scatter(
    df["temp_c"],
    df["pm25"]
)


plt.title("Temperature vs PM2.5")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()
```

Expected Output: Three charts are displayed: Daily PM2.5 Trend, Station Comparison and Temperature vs PM2.5. The values depend on the contents of `cleaned_readings.csv`.

Alternative Concept and Complexity Analysis

The cleaning process can also be implemented using pandas.

Pandas provides functions such as `read_csv()`, `dropna()`, `to_numeric()` and `groupby()`.

Both approaches take approximately O(n) time for a single pass through n records.

Pandas is convenient for large-scale numerical analysis, while a CSV loop can process records incrementally without loading the entire file into memory.

TASK B2 – ENGINEER ADVERSARIAL DATA (5 Marks)

Bloom's Level: K4 – Analyse

Aim

To test the reliability and security of the data-processing system using malformed CSV files, incorrect delimiters, missing columns, unusual station names and SQL injection-like input.

Algorithm

1. Create sample CSV files containing different types of errors.
2. Read the CSV headers.
3. Check whether all required columns are available.
4. Reject files with missing columns.
5. Reject files with incorrect delimiters.
6. Reject empty files.
7. Insert valid data using parameterised SQL queries.
8. Test station names containing special characters.
9. Compare the result with the expected output.
10. Record the errors and observations.

Code

```
import csv
import io
import sqlite3

readings = [
    ("Adyar", "2026-10-01", 48.0),
    ("Guindy", "2026-10-01", 95.0),
    ("Guindy", "2026-10-03", 121.0),
    ("Adyar", "2026-10-04", 60.0)
]

con = sqlite3.connect(":memory:")

con.execute("""
```

```
CREATE TABLE aq(
    station TEXT,
    day TEXT,
    pm25 REAL
)
""")

con.executemany(
    "INSERT INTO aq VALUES (?,?,?)",
    readings
)

con.commit()


def station_average(name):

    row = con.execute(
        "SELECT AVG(pm25) FROM aq WHERE station=?",
        (name,)
    ).fetchone()

    return round(row[0], 1) if row[0] is not None else None


print(
    station_average("Adyar"),
    station_average("Guindy")
)

print(
    station_average("Adyar' OR '1'='1")
)


cases = {
    "missing_column":
        "station,date\nAdyar,2026-10-01\n",

    "wrong_delimiter":
        "station;date;pm25\nAdyar;2026-10-01;48\n",

    "empty": ""
}


for label, text in cases.items():

    try:

        reader = csv.DictReader(
            io.StringIO(text)
        )
```

```
        fields = reader.fieldnames or []

        required = {
            "station", "date", "pm25"
        }

        if not required.issubset(fields):
            raise ValueError(
                "required columns missing or delimiter incorrect"
            )

        print(label, "OK", list(reader))

    except ValueError as exc:

        print(
            label,
            "REJECTED:",
            exc)

print(
    "Safe quote search:",
    station_average("Adyar' OR '1'='1"))

con.close()
```

Output

```
54.0 108.0
None
missing_column REJECTED: required columns missing or delimiter incorrect
wrong_delimiter REJECTED: required columns missing or delimiter incorrect
empty REJECTED: required columns missing or delimiter incorrect
Safe quote search: None
```

Design, Analysis and Observation

B2.1 – Missing Column Test

A CSV file may not contain all required columns.

For example, a file may contain only station and date, without PM2.5.

The program checks the header before processing records. If a required column is missing, the file is rejected.

B2.2 – Wrong Delimiter Test

CSV files generally use commas to separate values.

However, some files may use semicolons.

The sample program checks whether the required column names are available. A semicolon-separated file read using the default comma delimiter is rejected.

B2.3 – Empty File Test

An empty file does not contain a valid header or data rows.

The program identifies this condition and rejects the file.

B2.4 – SQL Injection Test

SQL injection is a security problem that occurs when untrusted user input is directly added to an SQL query.

The program uses parameterised SQL:

```
con.execute(
    "SELECT AVG(pm25) FROM aq WHERE station=?",
    (name,))
```

The station name is treated as data rather than SQL code.

The unusual input:

```
Adyar' OR '1'='1
```

is treated as one station name. Since no station has that exact name, the query returns `None`.

B2.5 – Sentiment Testing

The sentiment system should also be tested using different complaint types.

Examples:

- "The air is clean today."
- "The air is dirty near the school."
- "The bus service is not helpful."
- "Great, another day of polluted air."

A simple dictionary-based sentiment model may not understand sarcasm or complicated sentence structures.

Alternative Concept and Complexity Analysis

Without parameterised queries, directly joining user input into an SQL statement can change the meaning of the query.

Parameterized queries prevent user input from being interpreted as SQL syntax.

For n records, batch insertion takes approximately O(n) time.

The use of `executemany()` reduces repeated Python function-call overhead compared with inserting every record separately.

TASK B3 – LOOPS VS LIBRARIES VS SQL (8 Marks)

Bloom's Level: K4 – Analyse

Aim

To compare the performance of Python loops, pandas and SQL for processing air-quality data. To measure execution time, verify result accuracy and identify the performance bottleneck using profiling tools.

Algorithm

1. Generate hourly air-quality readings for different numbers of stations.
2. Create datasets for 1, 12 and 120 stations.
3. Use Python loops to calculate daily averages.
4. Use pandas groupby to calculate the same averages.
5. Use SQLite GROUP BY to calculate the same averages.
6. Measure the execution time of each method.
7. Compare the output values to verify correctness.
8. Use cProfile to identify expensive operations.
9. Record the results and compare the methods.

Code

```
import csv
import io
import sqlite3
import time
import cProfile
import pstats

from collections import defaultdict
import pandas as pd
```

```
DAYS = 365 * 24


# Generate sample air-quality data
def make_csv(stations):

    out = io.StringIO()
    w = csv.writer(out)

    w.writerow(["station", "day", "pm25"])

    for s in range(stations):
        for h in range(DAYS):

            day = h // 24

            value = float(
                (s * 13 + h % 97) % 180
            )

            w.writerow([
                f"S{s+1}",
                day,
                value
            ])

    return out.getvalue()


# Method 1: Using Python loops
def loop_means(text):

    sums = defaultdict(float)
    counts = defaultdict(int)
```

```
    for row in csv.DictReader(io.StringIO(text)):

        key = (
            row["station"],
            int(row["day"])
        )

        sums[key] += float(row["pm25"])
        counts[key] += 1

    return {
        k: sums[k] / counts[k]
        for k in sums
    }


# Method 2: Using Pandas
def pandas_means(text):

    frame = pd.read_csv(io.StringIO(text))

    result = frame.groupby(
        ["station", "day"]
    )["pm25"].mean()

    return result.to_dict()


# Method 3: Using SQL
def sql_means(text):

    con = sqlite3.connect(":memory:")

    con.execute("""
        CREATE TABLE r(
            station TEXT,
            day INTEGER,
```

```
            pm25 REAL
        )
    """)

    rows = csv.DictReader(io.StringIO(text))

    con.executemany(
        "INSERT INTO r VALUES (?,?,?)",
        (
            (
                x["station"],
                int(x["day"]),
                float(x["pm25"])
            )
            for x in rows
        )
    )

    result = {
        (station, day): avg
        for station, day, avg in con.execute("""
            SELECT station, day, AVG(pm25)
            FROM r
            GROUP BY station, day
        """).fetchall()
    }

    con.close()

    return result


# Compare performance of all three methods
for stations in (1, 12, 120):

    print("\nTesting", stations, "stations")
```

```
text = make_csv(stations)

times = {}
results = {}

methods = [
    ("loop", loop_means),
    ("pandas", pandas_means),
    ("sql", sql_means)
]

for name, fn in methods:

    start = time.perf_counter()

    results[name] = fn(text)

    times[name] = (
        time.perf_counter() - start
    )

# Compare all calculated averages
agree = (
    results["loop"].keys()
    == results["pandas"].keys()
    == results["sql"].keys()
)

if agree:
    agree = all(
        abs(
            results["loop"][k]
            - results["pandas"][k]
        ) < 1e-9
        and
        abs(
            results["loop"][k]
```

```
                - results["sql"][k]
            ) < 1e-9
            for k in results["loop"]
        )

    print(
        stations,
        "stations,",
        stations * DAYS,
        "rows; groups:",
        len(results["loop"]),
        "agree:",
        agree,
        "seconds:",
        {
            k: round(v, 3)
            for k, v in times.items()
        }
    )


# Profile Python loop performance
print("\nProfiling Python loop method")

sample = make_csv(12)

prof = cProfile.Profile()

prof.enable()

loop_means(sample)

prof.disable()

pstats.Stats(
    prof
).sort_stats("cumulative").print_stats(8)
```

Output

```
Testing 1 stations
1 stations, 8760 rows; groups: 365 agree: True seconds: {'loop': 0.007, 'pandas': 0.002, 'sql': 0.013}

Testing 12 stations
12 stations, 105120 rows; groups: 4380 agree: True seconds: {'loop': 0.097, 'pandas': 0.015, 'sql': 0.161}

Testing 120 stations
120 stations, 1051200 rows; groups: 43800 agree: True seconds: {'loop': 0.955, 'pandas': 0.146, 'sql': 1.678}

Profiling Python loop method
         630775 function calls (630774 primitive calls) in 0.169 seconds

   Ordered by: cumulative time
   List reduced from 21 to 8 due to restriction <8>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        3    0.018    0.006    0.225    0.075 /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/idle
lib/rpc.py:383(pollmessage)
   105121    0.076    0.000    0.120    0.000 /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/csv.
py:112(__next__)
        3    0.000    0.000    0.102    0.034 /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/idle
lib/rpc.py:352(pollpacket)
        2    0.018    0.009    0.061    0.031 {built-in method select.select}
      2/1    0.000    0.000    0.061    0.061 /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/idle
lib/rpc.py:397(pollresponse)
        1    0.012    0.012    0.040    0.040 /Users/janyaa/Desktop/1St YEAR/Python/ass5.py:633(loop_means)
   105122    0.020    0.000    0.020    0.000 {built-in method builtins.next}
   210241    0.014    0.000    0.014    0.000 /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/csv.
py:98(fieldnames)
```

The number of records generated by the program is:

Number of Stations	Records per Station	Total Records	Daily Groups
1	8, 760	8, 760	365
12	8, 760	105, 120	4, 380
120	8, 760	1, 051, 200	43, 800

The program prints the measured execution time for each method.

Expected correctness result:

```
1 stations, 8760 rows; groups: 365; agree: True

12 stations, 105120 rows; groups: 4380; agree: True

120 stations, 1051200 rows; groups: 43800; agree: True
```

The exact execution times and cProfile results depend on the computer and software environment. The actual measured values should be copied from the program output after running it.

Design, Analysis and Observation

B3.1 – Python Loops

Python loops process each record individually.

A dictionary stores the sum and count for every station and date.

The average is calculated by dividing the sum by the number of records.

Advantages:

- Easy to understand.
- Does not require pandas.
- Can process records incrementally.
- Useful for small datasets.

Disadvantages:

- Repeated Python-level operations can become slow for large datasets.
- The programmer must manage grouping and calculations manually.

B3.2 – Pandas

Pandas provides vectorised operations and groupby functionality.

The `groupby()` method groups records based on station and date.

The `mean()` function calculates the average for each group.

Advantages:

- Short and readable code.
- Useful for data analysis.
- Provides many built-in statistical functions.
- Convenient for handling missing values and transformations.

Disadvantages:

- Data is generally loaded into memory.
- Large DataFrames may require significant memory.

B3.3 – SQL

SQLite stores readings in a database table.

The SQL query uses:

```
SELECT station, day, AVG(pm25)
FROM r
GROUP BY station, day;
```

Advantages:

- Suitable for structured data.
- Supports repeated queries.
- Supports indexes and constraints.
- Can store data on disk.

Disadvantages:

- Requires database setup.
- Database operations have insertion and query overhead.
- SQL may be less convenient for some exploratory data-science operations.

B3.4 – Performance Comparison

The three approaches calculate the same daily averages.

Their performance depends on:

- Number of records.
- Available memory.
- CPU speed.
- Python version.
- Pandas version.
- SQLite configuration.
- Database insertion method.

The execution time must be measured on the same computer to make a fair comparison.

B3.5 – Profiling

The `cProfile` module identifies functions that consume more execution time.

In the loop implementation, CSV parsing, float conversion and dictionary updates are possible performance bottlenecks.

The profiling results should be inspected to identify the actual expensive operations.

Alternative Concept and Complexity Analysis

The Python dictionary method takes approximately O(n) expected time for n records.

It uses O(g) additional memory, where g is the number of station-date groups.

Pandas also processes the records in approximately O(n) time for grouping, but generally requires memory for the DataFrame.

SQL performs aggregation inside the database and can use indexes to improve queries.

For small datasets, Python loops may be sufficient. For exploratory analysis, pandas is convenient. For repeated reports and persistent storage, SQL is suitable.

Benchmark Note: Run the program at least three times on the same computer and report the median timing. Do not use invented benchmark values because timings depend on the execution environment.

TASK B4 – FINAL ENGINEERING DECISION (5 Marks)

Bloom's Level: K5 – Evaluate

Aim

To evaluate the sentiment-analysis system using at least 50 citizen complaints, compare database query performance before and after indexing, and recommend a suitable architecture for a larger air-quality monitoring network.

Algorithm

1. Prepare a set of citizen complaints.
2. Apply the sentiment-scoring function to every complaint.
3. Assign reference labels for evaluation.
4. Compare predicted labels with reference labels.
5. Calculate the agreement percentage.
6. Create a SQLite database containing a larger set of readings.
7. Execute repeated station/date queries.
8. Record the execution time before indexing.
9. Create a composite index on station and date.

10. Repeat the same query workload.

11. Compare the results.

12. Recommend suitable technologies for the expanded network.

Code

```
import re
import sqlite3
import time

POSITIVE = {
    "good", "clean", "fast",
    "helpful", "improved"
}

NEGATIVE = {
    "bad", "dirty", "slow",
    "rude", "smoke", "broken"
}

NEGATORS = {
    "not", "never", "no"
}


def score(text):

    words = re.findall(
        r"[a-z]+",
        text.lower()
    )

    total = 0

    for i, word in enumerate(words):

        value = (
            1 if word in POSITIVE
            else -1 if word in NEGATIVE
            else 0
        )

        if i > 0 and words[i-1] in NEGATORS:
            value *= -1

        total += value

    return total


def label(text):
```

```
    value = score(text)

    if value > 0:
        return "positive"

    if value < 0:
        return "negative"

    return "neutral"


complaints = [
    "Roads are clean and the bus was fast.",
    "Smoke from the factory, air is bad.",
    "The staff were not helpful.",
    "Not bad, service improved.",
    "The air is dirty near the school.",
    "The bus service is slow.",
    "The repair team was helpful.",
    "The road is clean but traffic is slow.",
    "No smoke today and the air is good.",
    "The sensor is broken and the report is bad."
]

complaints = (complaints * 5)[:50]

labels = [
    label(x)
    for x in complaints
]

manual = []

for x in complaints:

    text = x.lower()

    if (
        "clean" in text or
        "helpful" in text or
        "improved" in text or
        "not bad" in text
    ):
        manual.append("positive")

    elif (
        "bad" in text or
        "dirty" in text or
        "smoke" in text or
        "slow" in text or
        "broken" in text or
        "not helpful" in text
    ):
        manual.append("negative")
```

```
    else:
        manual.append("neutral")


agree = sum(
    a == b
    for a, b in zip(labels, manual)
)

print("Complaints:", len(complaints))

print(
    "Agreement:",
    agree,
    "/",
    len(complaints),
    "=",
    round(100 * agree / len(complaints), 1),
    "%"
)

for text, tag in zip(
    complaints[:4],
    labels[:4]
):
    print(tag, "|", text)


con = sqlite3.connect(":memory:")

con.execute("""
CREATE TABLE readings(
    station TEXT,
    day TEXT,
    pm25 REAL
)
""")

rows = [
    (
        f"S{i % 120}",
        f"2026-10-{i % 30 + 1:02d}",
        float(i % 150)
    )
    for i in range(200000)
]

con.executemany(
    "INSERT INTO readings VALUES (?,?,?)",
    rows
)

query = """
```

```
SELECT AVG(pm25)
FROM readings
WHERE station=? AND day=?
"""

params = ("S17", "2026-10-18")

start = time.perf_counter()

for _ in range(30):
    con.execute(
        query,
        params
    ).fetchone()

before = time.perf_counter() - start

con.execute("""
CREATE INDEX idx_station_day
ON readings(station, day)
""")

start = time.perf_counter()

for _ in range(30):
    con.execute(
        query,
        params
    ).fetchone()

after = time.perf_counter() - start

print(
    "Query seconds before index:",
    round(before, 4)
)

print(
    "Query seconds after index:",
    round(after, 4)
)

con.close()
```

Output

```
Complaints: 50
Agreement: 35 / 50 = 70.0 %
positive | Roads are clean and the bus was fast.
negative | Smoke from the factory, air is bad.
negative | The staff were not helpful.
positive | Not bad, service improved.
Query seconds before index: 0.1651
Query seconds after index: 0.0105
```

The agreement count, percentage and query times are calculated and displayed when the program is executed. They should be copied from the actual run.

Design, Analysis and Observation

B4.1 – Sentiment Evaluation

The program tests the sentiment system using 50 complaint entries.

Each complaint is classified as positive, negative or neutral.

The predicted labels are compared with the reference labels.

The agreement percentage is calculated using:

Agreement (%) = (Correct Predictions / Total Complaints) × 100

This provides a simple measurement of how often the sentiment system agrees with the reference labels.

For a realistic evaluation, the dataset should contain at least 50 different complaints labelled by a human reviewer. The repeated sample phrases in this demonstration show how the evaluation procedure works, but they are not a representative sample of all citizen complaints.

B4.2 – Limitations of Sentiment Analysis

The dictionary-based approach has several limitations.

1. It may not correctly understand sarcasm.
2. It may fail to recognise spelling mistakes.
3. It has limited understanding of sentence context.
4. It may misclassify mixed positive and negative opinions.
5. It may not work accurately with multiple languages.

For example, a complaint such as "Great, another day of polluted air" may be incorrectly classified because the program does not understand sarcasm.

A trained model using labelled complaints may provide better results.

B4.3 – Database Indexing

An index is a database structure used to improve the speed of data retrieval.

The program creates a composite index:

```
CREATE INDEX idx_station_day
ON readings(station, day);
```

The index helps SQLite locate records matching a station and date.

The same query is executed 30 times before and after creating the index.

The measured times are printed by the program.

An index may improve query performance, but it also requires additional storage and can increase the cost of inserting and updating records.

B4.4 – Production Design

For a larger network with 112 monitoring stations, each station reports every 15 minutes.

Each station produces:

96 readings per day.

Total readings per day:

112 × 96 = 10,752 readings.

Approximate readings per year:

10,752 × 365 = 3,924,480 readings.

This is a large dataset and requires a reliable storage and processing system.

Recommended Production Architecture

1. **Data Acquisition:** Collect readings through scheduled station uploads or an ingestion service.
2. **Raw Data Storage:** Preserve original CSV files for auditing and recovery.
3. **Data Cleaning:** Validate missing values, invalid measurements, duplicates and units.
4. **Database Storage:** Use a relational database such as PostgreSQL for concurrent users and larger operational workloads.

5. **Data Analysis:** Use SQL for repeated summaries and pandas for exploratory analysis.
6. **Visualisation:** Use dashboards and charts to display trends and station comparisons.
7. **Complaint Processing:** Use sentiment analysis to classify complaints and identify important issues.
8. **Security:** Use parameterised queries, access control and secure storage.
9. **Monitoring:** Track data quality, processing failures and application performance.
10. **Evaluation:** Regularly test the sentiment model against human-labelled complaints.

Alternative Concept and Complexity Analysis

A more advanced sentiment system can use a machine-learning classifier trained on previously labelled citizen complaints.

The dictionary-based approach takes approximately O(w) time for a complaint containing w words.

A frequency-based extractive summariser also processes the words and sentences to identify important information.

A trained model requires more computing resources, but it may better understand context and negation.

The final production decision should be based on measured accuracy, response time, memory usage, operating cost and the ability to explain predictions.
